RRST
Ritsumeikan Robot research Society of Technology

serial_bridge 講習

ROS 2 ⇔ マイコン シリアル通信パッケージ

立命館大学 ロボット技術研究会 RRST NHK プロジェクト

はじめに

serial_bridge の後継として ros2can を開発しました。

今後 serial_bridge のバグ修正・機能追加の予定はありません。新規の機体では ros2can の採用を強く推奨します。

この講習の位置づけ

1 既存機体の保守

serial_bridge で動いている機体を読んで・直せるようになる

2 設計思想を知る

ID 方式・仲介ノード・24 スロットのフレームは ros2can にもそのまま引き継がれている

3 移行の準備

違いがわかれば ros2can への移行がスムーズになる

serial_bridge とは

ROS 2 と複数マイコンの**双方向通信**を実現する軽量・高速な通信パッケージ。micro-ROS と比べて容量・動作が軽く、ロボコンに特化しています。学ロボ 26 向けに開発しましたが、機体に依存せず使えます。

24
× int16 スロット
（送信・受信それぞれ）

115200
bps の USB シリアル

自動
ポートスキャン
（ポート指定不要）

ホットプラグ
抜き挿ししても
自動で復帰

- マイコンはフレーム内の DEVICE_ID で識別するので、USB ポートの順番に依存しない
- XOR チェックサムでフレームを検証、切断・タイムアウト時は自動で再接続
- ESP32 / STM32 / B-G431 等のファームウェアを同梱（ PlatformIO ）。ID とモードを設定して書き込むだけ

RRST
Ritsumeikan Robot research Society of Technology

01

開発背景と要件

なぜシリアル通信 + ESP32 なのか

開発背景：無線から有線へ

学ロボ 25
UDP 経由の無線通信

→

問題
電波干渉で会場での通信が不安定

→

学ロボ 26
有線通信の確立を目標に

有線通信の選択肢

EtherCAT

ロボコンの主流の 1 つ

導入コストが大きく、予算の少ないプロジェクトでは困難

CAN

ロボコンの主流の 1 つ

導入コストは低いが、CAN 対応 MD や各アクチュエータへの変換など開発コストが高い

シリアル通信

採用

講習でも扱っており、追加のハードウェアやライブラリなしで実装できる

要件の設定

1 回生ではじめてロボットを触る人が大多数。システムの維持・開発を極力簡単にするため、次の要件を定めました。

1. **開発者 1 名で開発できる**
2. **低予算で実装できる**
3. **追加のハードウェアが少ない（既存のハードを活用）**
4. **高速である**
5. **軽量である**
6. **学習コストが低い（できれば Arduino 言語）**
7. **ROS に依存しない**
8. **有線通信である**
9. **汎用性が高い（他の機体でも流用）**

仕様の決定

通信方式

シリアル通信

- 講習でも扱っている
- 追加のハードウェア・ライブラリが不要

マイコン

ESP32

- Arduino 言語で扱える
- 速度・ピン数も問題なし
- STM32 は主流だが CubeIDE の学習コストが高く断念

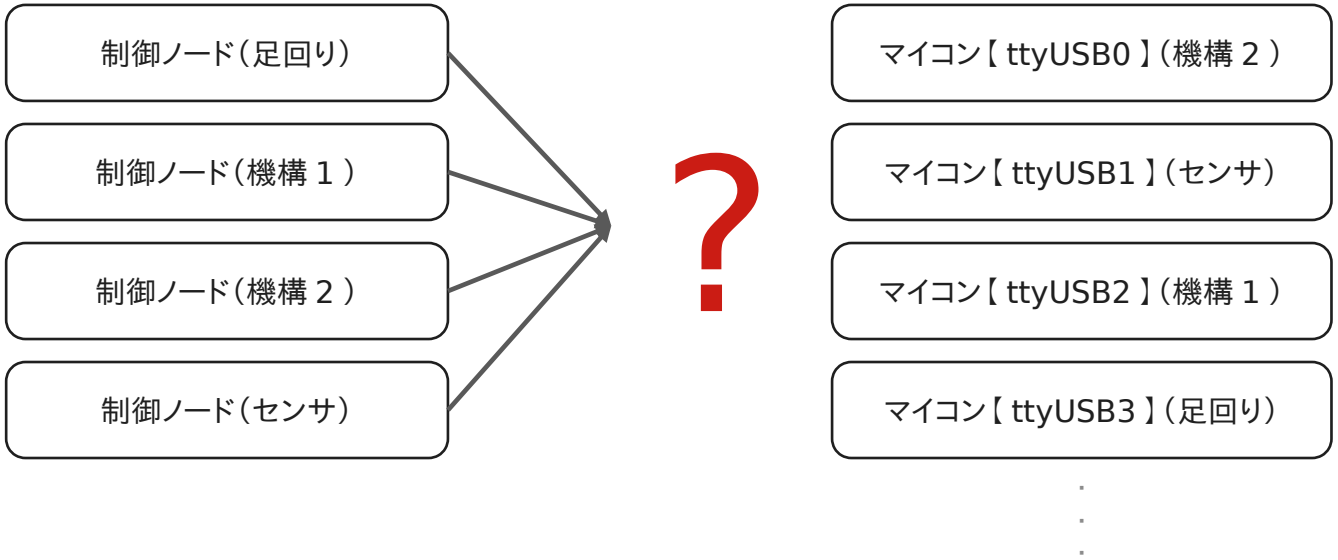
設計方針: 通信部分は特定のマイコンに依存しないシンプルな設計に。
→ 後に STM32 / B-G431（SimpleFOC）版のファームウェアや、ros2can への拡張ができた土台になっています。

RRST
Ritsumeikan Robot research Society of Technology

02
システム設計

ポート番号問題と、ID 方式・仲介ノードという解決策

課題：マイコンが識別できない



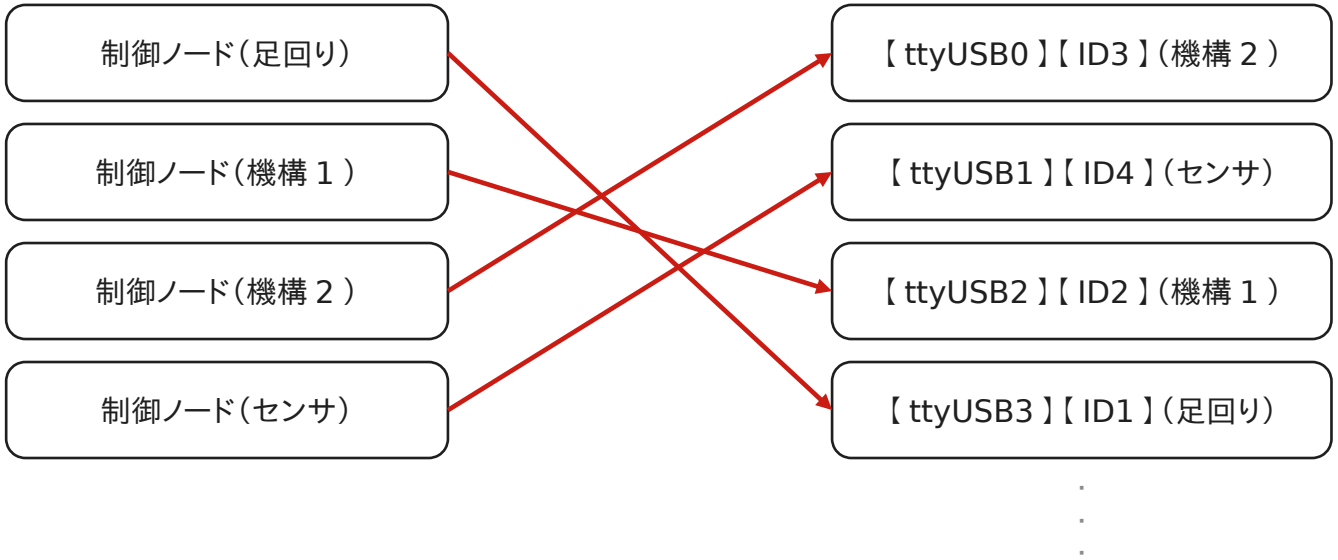
USB に挿した順番にポート番号（ ttyUSB の末尾）が割り当てられる

ノード側でポートを決め打ちしても、挿す順番が変わると通信できない

電源を入れるたびに順番通り挿し直すのは、セッティングタイムやリトライ中には現実的ではない

→ どのポートにどのマイコンが挿さっているかを、**ソフト側で判別する仕組み**が必要

解決①:マイコンに ID を持たせる



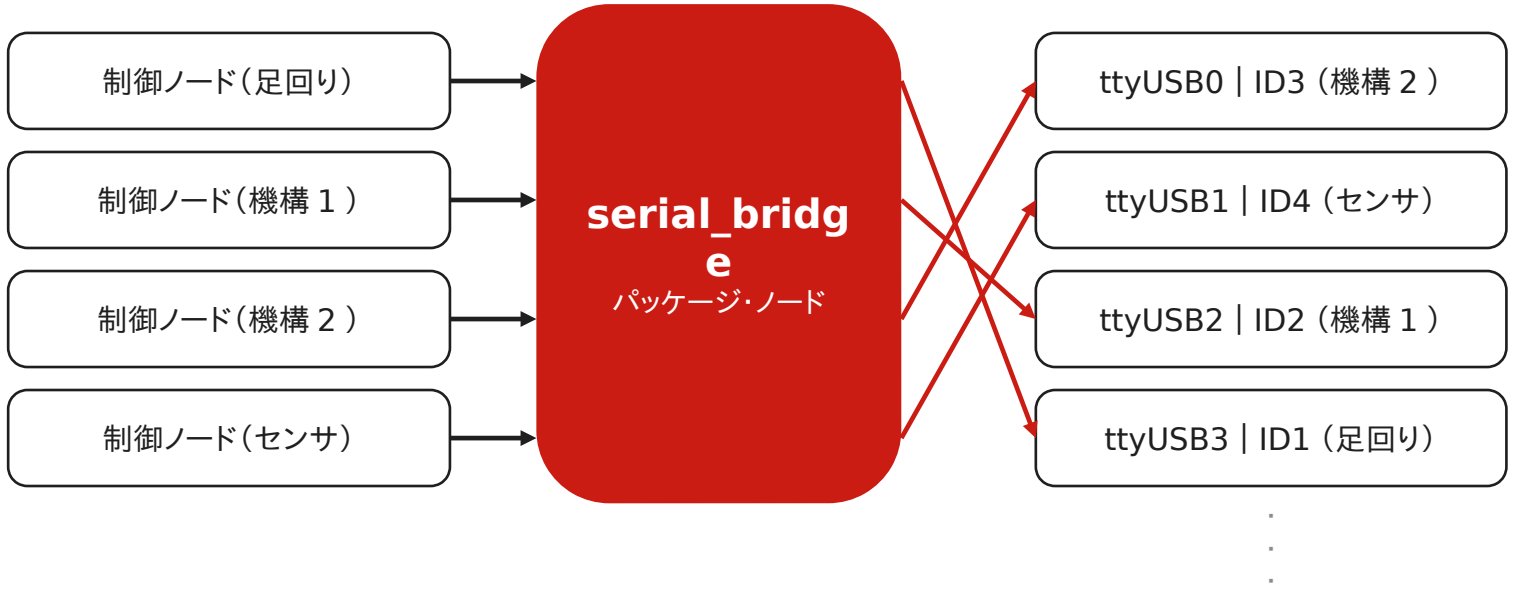
ID はマイコンのプログラムに予め保持

マイコンは常に ID を載せたフレームを PC へ送信し続ける

ノード側が宛先マイコンの ID を知っていれば、ポート番号に関係なく通信できる

まだ問題が残る:各ノードが「ポートを開く → ID を確認 → 違えば閉じて次へ」を個別に行うと、ポートのオープン・クローズのタイミングが干渉して正常に動かない

解決②：ポート操作を仲介するノード



ポートを触るのは

serial_bridge だけ

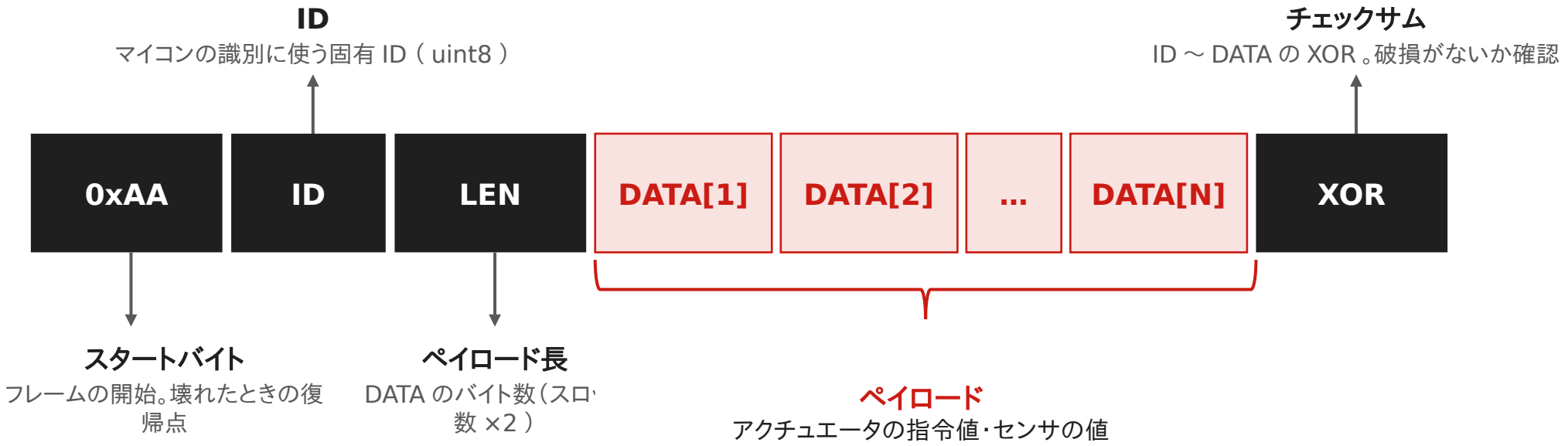
各ノードは serial_tx_[ID] に Publish するだけ

スキャンと宛先 ID への送信は serial_bridge が担当

名前の由来：シリアルポートとの「橋渡し」なので serial_bridge

この「ポートを専有する主体を 1 つにまとめる」設計は ros2can にもそのまま引き継がれています。

通信フレームの定義



ポイント: 値はノード上では int16 、シリアルでは 8bit × 2 に分割して送る(上位バイトが先=ビッグエンディアン)。フレームにペイロードを載せて送受信するイメージです

ペイロードの割当（esp32_serial_bridge）

RRST
Ritsumeikan Robot research Society of Technology

ROS 2 → マイコン（serial_tx）

[N]	用途	範囲
0	DEBUG	0 / 1
1 ～ 8	DC モータの指令値	-255 ～ 255
9 ～ 16	PWM サーボの指令値	0 ～ 270
17 ～ 23	トランジスタアレイ（TR）	0 / 1

マイコン → ROS 2（serial_rx）

[N]	用途
1 ～ 8	エンコーダのパルス
9 ～ 16	マイクロスイッチ（デジタル入力）
17 ～ 23	その他センサのために予約

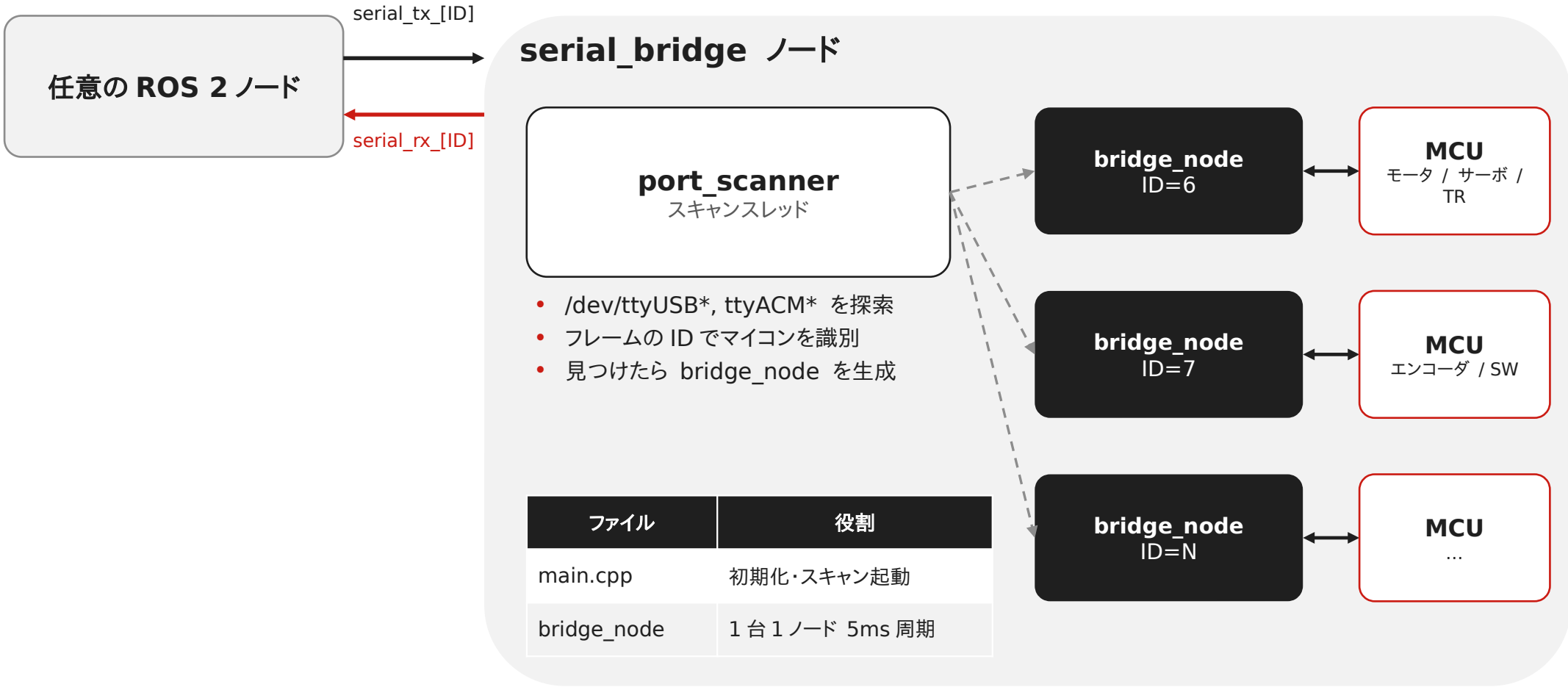
- スロット数は 24（config.hpp で変更可能）。1 スロット = int16 = 2 バイトなので、ペイロードは 48 バイト
- 割当はファームウェアごとに固定。どのスロットが何を意味するかを serial_bridge 自体は解釈しない
- 範囲を超えないよう、自作ノードでは PacketController（後述）を使うと安全

RRST
Ritsumeikan Robot research Society of Technology

03
実装

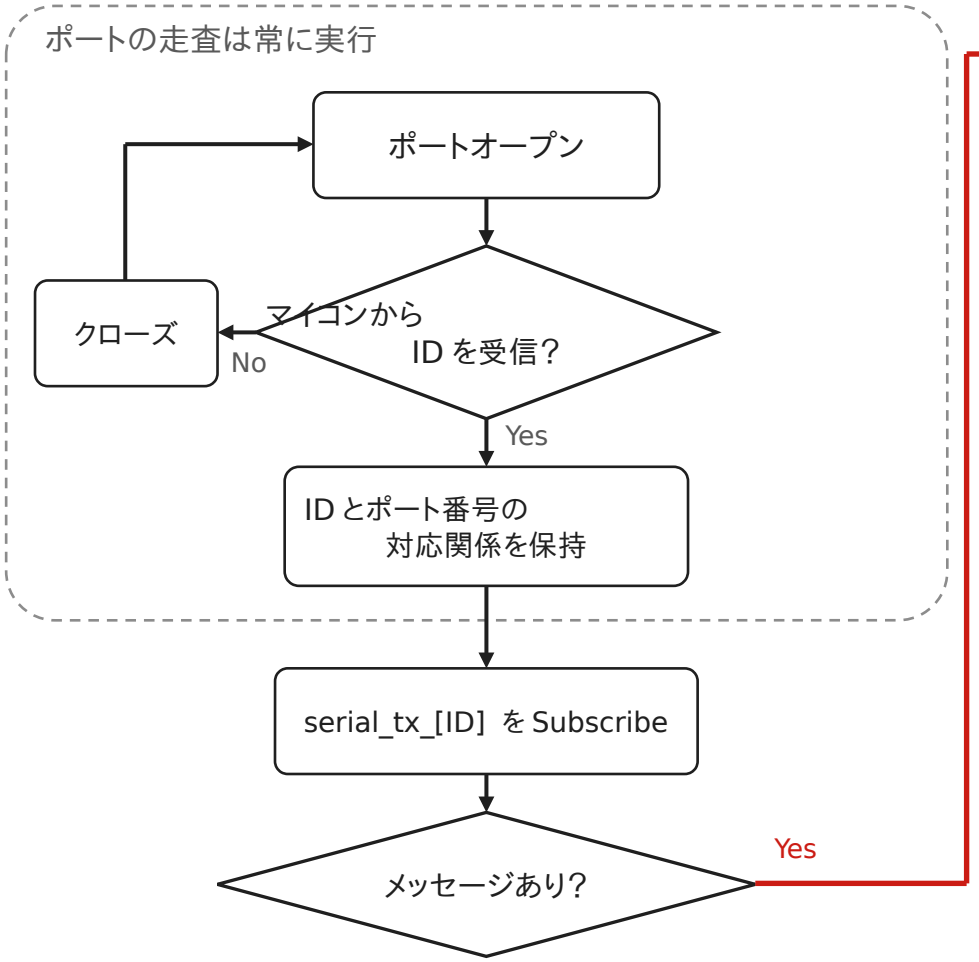
ROS 2 側の処理とマイコン側の処理

ROS 2 側の構成



ファイル	役割
main.cpp	初期化・スキャン起動
bridge_node	1 台 1 ノード 5ms 周期

送信処理の流れ（ROS 2 → マイコン）



1 **int16 → uint8 配列に変換**

これをペイロードとする

2 **フレーム組み立て**

スタートバイト・ID・ペイロード長・ペイロード・チェックサムの順に結合

3 **ID に対応したポートへ送信**

write() で書き込み

受信も同様の流れで serial_rx_[ID] に Publish

ホットプラグと自動再接続

マイコンを挿したとき

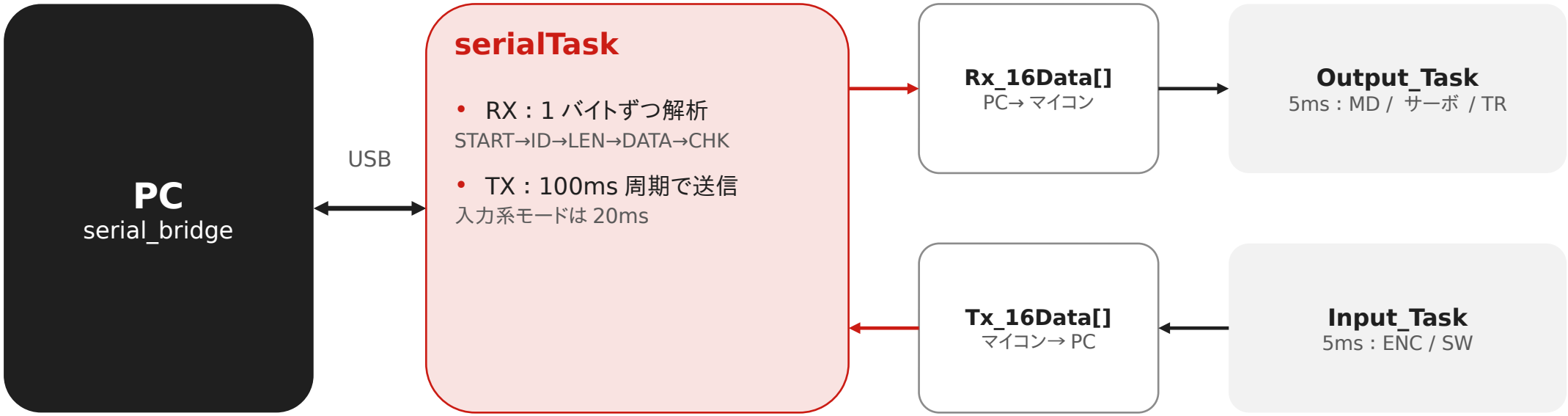
1. port_scanner が /dev/ttyUSBx で有効なフレームを検出
2. bridge_node を生成して executor に追加
3. serial_rx_[ID] / serial_tx_[ID] が有効に

抜けた・タイムアウトしたとき

1. EIO / ENODEV / ENXIO、または rx_timeout_sec 経過を検出
2. ポートを閉じ、reconnect_interval_sec ごとに再オープンを試行
3. 別のポートで見つかったらノードを差し替え

Tips : 識別は ID で行うので、別の USB ポートに挿し直しても同じ serial_rx_[ID] / serial_tx_[ID] で通信できます。

マイコン側の構成（ESP32 / FreeRTOS）



- setup() で serialTask と MODE に応じた制御タスクを生成。loop() は何もしない（実処理は全て FreeRTOS タスク）
- 起動時に DEVICE_ID の回数だけ LED が点滅 → 目視で ID を確認できる
- 通信と入出力が共有配列を介して疎結合 → 通信部分だけを他マイコンへ移植しやすい

ファームウェアの MODE（ESP32 版）

MODE	用途	DC モータ	サーボ	TR	ロボマス	ENC	SW
MODE_OUTPUT	出力のみ	4	4	5	-	-	-
MODE_INPUT	入力のみ	-	-	-	-	4	8
MODE_IO	入出力混在	2	-	5	-	2	4
MODE_ROBOMAS	ロボマス制御	-	-	-	4	-	-
MODE_ROBOMAS_PLUS_OUTPUT	ロボマス ＋ 出力	4	4	5	4	-	-
MODE_ROBOMAS_PLUS_INPUT	ロボマス ＋ 入力	-	-	-	4	4	8
MODE_ROBOMAS_PLUS_IO	ロボマス ＋ 一部入出力	-	-	5	4	4	-

設定手順: src/config.hpp で DEVICE_ID を一意に設定 → MODE を 1 つだけ有効化 → 必要なら PWM・サーボ定数を調整 → PlatformIO で書き込み

RRST
Ritsumeikan Robot research Society of Technology

04
使い方

ビルド・起動・パラメータ・自作ノード

ビルド・起動・パラメータ

```
# ポート権限（初回のみ・要再ログイン）
sudo usermod -aG dialout $USER

# ビルド
cd ~/ros2_ws
colcon build --packages-select serial_bridge
source install/setup.bash

# 起動（すぐにスキャン開始）
ros2 launch serial_bridge serial_bridge.launch.py
```

ログ出力モード（config.hpp 、コンパイル時）

kTerminal	kGraphical（既定）	kNone
通常のログ	マイコンごとのバーグラフ	出力なし

config/serial_bridge.yaml

パラメータ	既定	内容
excluded_ports	[ttyUSB0]	スキャン除外（LiDAR 等）
rx_timeout_sec	2.0	無受信で切断とみなす秒数
reconnect_interval_sec	3.0	再接続の最小間隔
scan_interval_ms	5000	スキャン間隔

LiDAR など他のシリアル機器は excluded_ports に入れておかないと、スキャン時にポートを開いてしまいます。

自作ノードから使う

```
#include "common/PacketController.hpp"
PacketController ctrlPkt;

// 送信: serial_tx_[ID] に Publish するだけ
pub_ = create_publisher<Int16MultiArray>("serial_tx_3", 10);

void timer_callback() {
  ctrlPkt.setMD(MD1, 120);        // -255 ～ 255 にクランプ
  ctrlPkt.setServo(SERVO2, 90);   // 0 ～ 270
  ctrlPkt.setTR(TR5, true);
  Int16MultiArray msg;  msg.data = ctrlPkt.toVector();
  pub_->publish(msg);
}

// 受信: serial_rx_[ID] を Subscribe
void sensor_callback(const Int16MultiArray::SharedPtr m) {
  int16_t enc1 = m->data[1];      // エンコーダ 1
  bool    sw1  = m->data[9];      // スイッチ 1
}
```

PacketController

メソッド	範囲
setMD(MDn, v)	-255 ～ 255
setServo(SERVOn, v)	0 ～ 270
setTR(TRn, on)	0 / 1
toVector()	送信用配列

- 範囲外の値は自動でクランプ
- ID はマイコンの config.hpp の DEVICE_ID
- 受信確認は ros2 topic echo /serial_rx_3

まとめ: ros2can へ

ID 方式	ポート番号ではなくフレーム内の ID でマイコンを識別
仲介ノード	ポートを触るのは serial_bridge だけ。各ノードはトピックで通信
フレーム	0xAA・ID・LEN・24×int16・XOR のシンプルな形式
疎結合 FW	通信タスクと入出力タスクが共有配列で連携

次のステップ: ros2can

- これらの設計はすべて ros2can に引き継がれている
- CAN デイジーチェーンで、PC との USB は 1 本に
- GUI ・仮想デバイス・ FW 生成を搭載
- serial_tx_[ID] / serial_rx_[ID] はそのまま

GitHub : github.com/RRST-NHK-Project/serial_bridge ／ 後継: github.com/RRST-NHK-Project/ros2can